NAZARBAYEV UNIVERSITY

Numerical Methods with Applications
Math 351 – 1L, 2L

Fall 2026

Chapter 3

August 21, 2026

Roots of Continuous Functions

- Given a continuous function $f = f(x)$, consider the problem

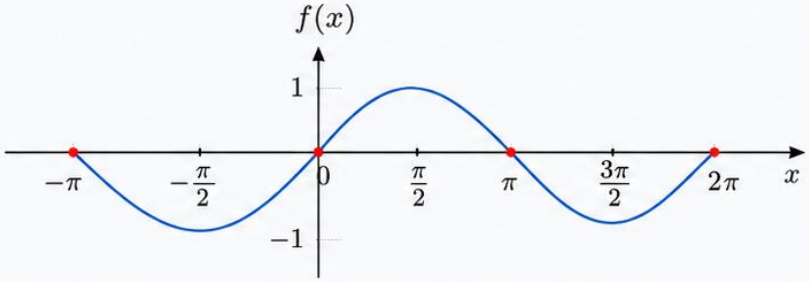
$$f(x) = 0,$$

  where x is the independent variable in the interval $[a, b]$.

- If x^* is a value of x where f vanishes, i.e., $f(x^*) = 0$, then x^* is called a **root** or **zero** of f.

- *How many roots?* This depends not only on the function but also on the interval.

Example: $f(x) = \sin x$



One root in	Two roots in	No roots in
$[-\pi/2, \pi/2]$	$[-\pi/4, 5\pi/4]$	$[\pi/4, 3\pi/4]$

Why study a nonlinear problem before addressing linear ones?

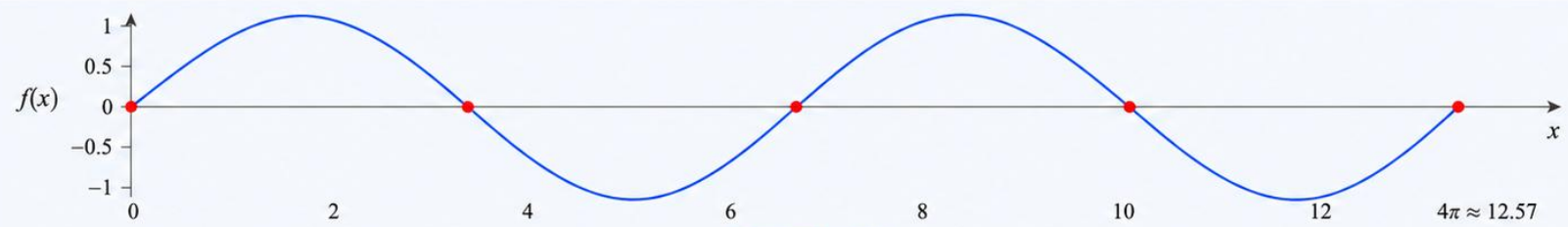
ax=b

One linear equation, e.g. $ax = b$, is too simple (solution: $x = b/a$, duh!), whereas a system of linear equations (Chapters 5 and 7) can have many complications.

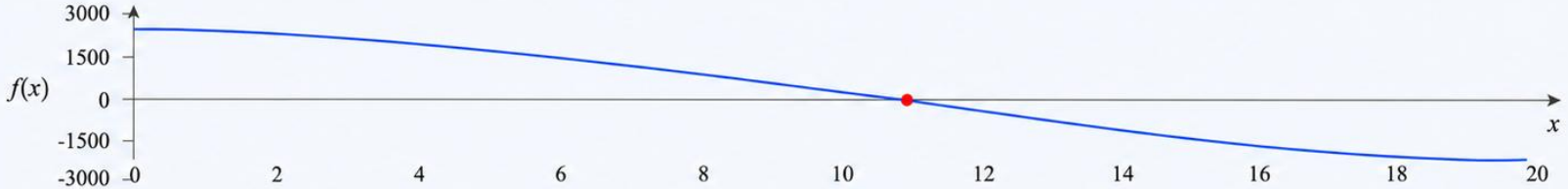
Several important general methods (e.g., bisection, fixed-point, Newton's method) can be developed and understood in a simple scalar context.

Several important algorithm properties (e.g., convergence, stability, robustness, efficiency) can be defined, illustrated, and used in a simple setting.

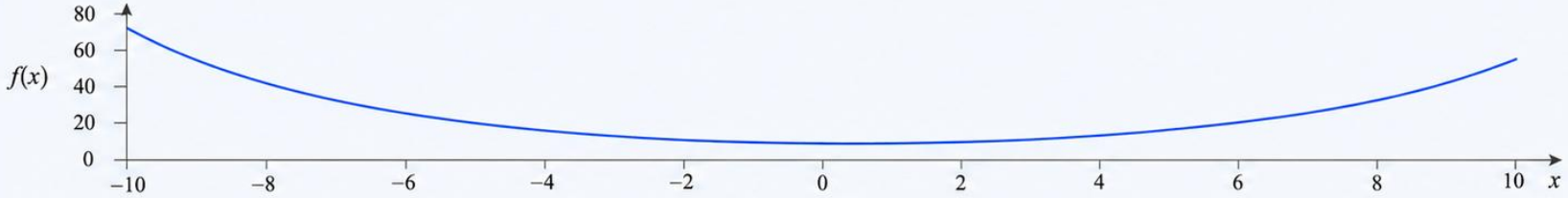
Examples of functions and roots



Several roots



One root

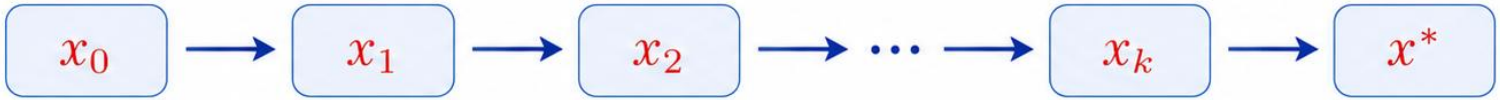


No roots on this interval

The number of roots depends on both the function and the interval.

Desirable algorithm properties

For a nonlinear problem, we usually use an **iterative method**: starting with an initial iterate (guess) x_0, generate iterates $x_1, x_2, \ldots, x_k, \ldots$ that hopefully converge to a root x^*.



Desired properties of an iterative method

- **Efficient:** requires a small number of function evaluations.
- **Robust:** fails rarely, if ever; announces failure if it does fail.
- **Requires minimal additional information**, such as the derivative of f.
- **Requires** f to satisfy only minimal smoothness properties.
- **Generalizes easily and naturally** to many equations in many unknowns.

Like many wish-lists, this one is hard to fully satisfy...

Outline

- Bisection method
- Fixed point iteration
- Newton's method and variants
- Minimizing a function in one variable

Bisection method

Advantages

- Simple
- Safe, robust
- Requires only that f be continuous

Limitations

- Slow
- Hard to generalize to systems

Bisection Method

1 Start with an interval

$[a_0, b_0]$ where $f(a_0)f(b_0) < 0$.

2 Compute the midpoint

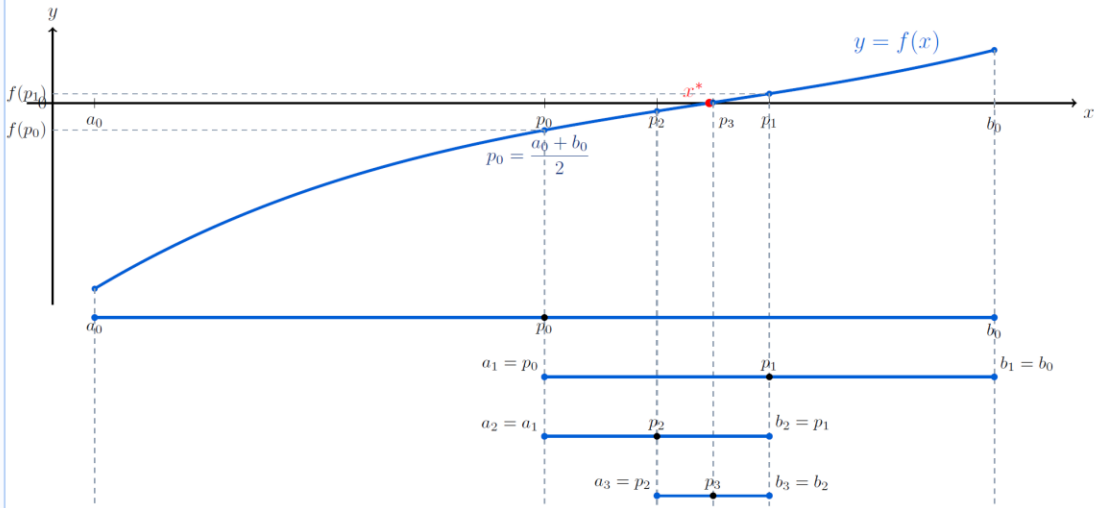
$$p_0 = \frac{a_0 + b_0}{2}.$$

3 Keep the half-interval

where the sign change remains.

4 Repeat

until the interval is sufficiently small.



Result

Nested intervals shrink to the root.

Nested intervals

$$[a_0, b_0] \supset [a_1, b_1] \supset [a_2, b_2] \supset [a_3, b_3] \supset \cdots$$

Bisection method development

1. Given $a < b$ such that $f(a) \cdot f(b) < 0$, there must be a root in $[a, b]$.
   Refer to $[a, b]$ as the **uncertainty interval**.

2. **So, at each iteration, evaluate $f(p)$ at $p = \frac{a+b}{2}$.**
   Check the sign of $f(a) \cdot f(p)$.
   - If positive, set $a \leftarrow p$;
   - if negative, set $b \leftarrow p$.

   Note:
   only one evaluation of the function f per iteration.

3. This reduces the length of the uncertainty interval by factor 0.5 at each iteration.
   So, setting $x_n = p$, the error after n iterations satisfies

$$\left|x^* - x_n\right| \leq \frac{b-a}{2} \cdot 2^{-n}.$$

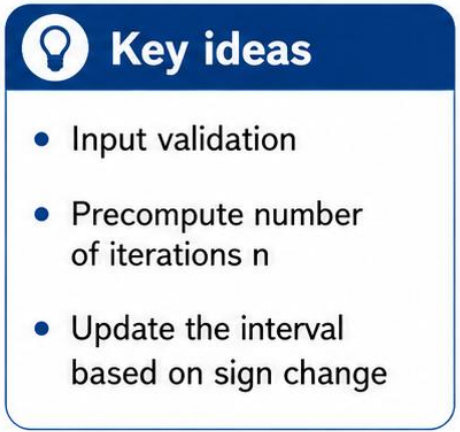
4. **Stopping criterion:** given (absolute) tolerance `atol`, require $\frac{b-a}{2} \cdot 2^{-n} \leq \texttt{atol}.$

5. This allows ***a priori*** determination of the number of iterations n: unusual in algorithms for nonlinear problems.

Bisection function

```
function [p,n] = bisect(func,a,b,fa,fb,atol)
if (a >= b) | (fa*fb >= 0) | (atol <= 0)
    disp('something wrong with the input: quitting');
    p = NaN; n = NaN;
    return
end

n = ceil( log2(b-a) - log2(2*atol) );
for k = 1:n
    p  = (a+b)/2;
    fp = feval(func,p);
    if fa * fp < 0
        b  = p;
        fb = fp;
    else
        a  = p;
        fa = fp;
    end
end
p = (a+b)/2;
```



Example Bisection Method



Show that $f(x) = x^3 + 4x^2 - 10 = 0$ has a root in $[1, 2]$, and use the Bisection method to determine an approximation to the root that is accurate to at least within 10^{-4}.

Example Bisection Method

Show that $f(x) = x^3 + 4x^2 - 10 = 0$ has a root in $[1, 2]$, and use the Bisection method to determine an approximation to the root that is accurate to at least within 10^{-4}.

1. Verify sign change

$f(1) = 1 + 4 - 10 = -5 < 0$

$f(2) = 8 + 16 - 10 = 14 > 0$

$\Rightarrow$ By IVT, a root exists in $[1, 2]$.

2. Bisection rule

At each step, compute $p_n = \frac{a_n + b_n}{2}$.

- If $f(a_n)f(p_n) < 0$ set $b_n = p_n$.
- Otherwise set $a_n = p_n$.

3. Stopping criterion

Stop when $\frac{b_n - a_n}{2} \leq 10^{-4}$.

n	a_n	b_n	$p_n = \frac{a_n + b_n}{2}$	$f(p_n)$
1	1.0	2.0	1.5	2.375
2	1.0	1.5	1.25	−1.79687
3	1.25	1.5	1.375	0.16211
4	1.25	1.375	1.3125	−0.84839
5	1.3125	1.375	1.34375	−0.35098
6	1.34375	1.375	1.359375	−0.09641
7	1.359375	1.375	1.3671875	0.03236
8	1.359375	1.3671875	1.36328125	−0.03215
9	1.36328125	1.3671875	1.365234375	0.000072
10	1.36328125	1.365234375	1.364257813	−0.01605
11	1.364257813	1.365234375	1.364746094	−0.00799
12	1.364746094	1.365234375	1.364990235	−0.00396
13	1.364990235	1.365234375	**1.365112305**	−0.00194

Approximate root: $x \approx \mathbf{1.365112305}$ (accurate to at least within 10^{-4}).

Bisection Method: Number of Iterations

Problem: Determine the number of iterations necessary to solve $f(x) = x^3 + 4x^2 - 10 = 0$ with accuracy 10^{-3}, using $a_1 = 1$ and $b_1 = 2$.

Bisection Method: Number of Iterations

Problem: Determine the number of iterations necessary to solve $f(x) = x^3 + 4x^2 - 10 = 0$ with accuracy 10^{-3}, using $a_1 = 1$ and $b_1 = 2$.

1 Since the bisection error satisfies $|p_N - p| \leq \dfrac{b-a}{2^N}$, and here $b - a = 1$, write $|p_N - p| \leq 2^{-N}$.

2 To guarantee accuracy 10^{-3}, require $2^{-N} < 10^{-3}$.

3 Take base-10 logarithms: $\log_{10}(2^{-N}) < \log_{10}(10^{-3})$.

4 Therefore, $-N \log_{10} 2 < -3$, so $N > \dfrac{3}{\log_{10} 2} \approx 9.96$.

Conclusion: Hence, 10 iterations are sufficient to ensure an approximation accurate to within 10^{-3}.

Bisection Method

Bisection rule

Start with $[a_1, b_1] = [1, 2]$

$f(a_1) < 0 < f(b_1)$

At each iteration

$$p_n = \frac{a_n + b_n}{2}$$

Keep the half-interval where the sign changes.

Interval width

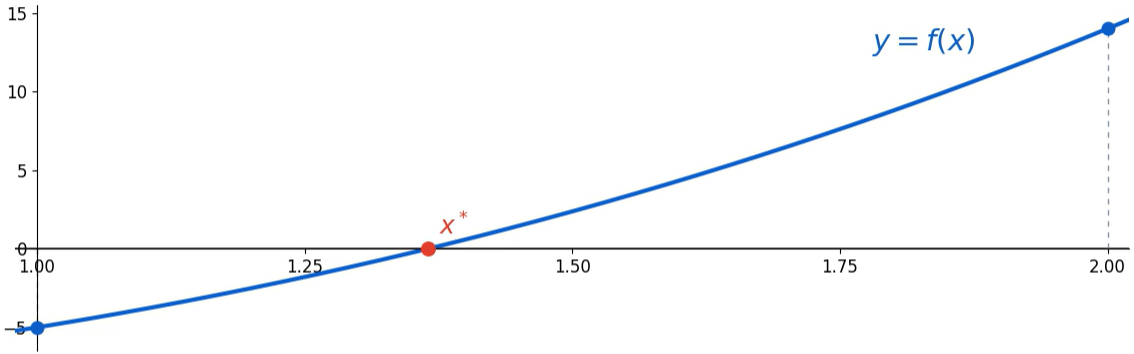
$$b_{n+1} - a_{n+1} = \frac{1}{2}(b_n - a_n)$$

Example

$f(x) = x^3 + 4x^2 - 10$

Find a root of $f(x) = x^3 + 4x^2 - 10$ in the interval $[1, 2]$.

$f(1) = -5 < 0, \quad f(2) = 14 > 0 \;\rightarrow\;$ a root lies in $[1, 2]$.



Initial uncertainty interval

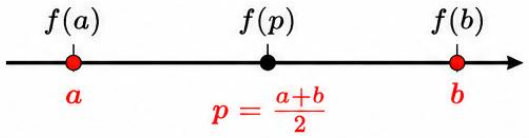


Bisection Method: Error Bound and Indexing

1 Given $a < b$ such that $f(a) \cdot f(b) < 0$, there must be a root in $[a, b]$.
Refer to $[a, b]$ as the **uncertainty interval**.

2 So, at each iteration, evaluate $f(p)$ at $p = \frac{a+b}{2}$.
Check the sign of $f(a) \cdot f(p)$.

- If positive, set $a \leftarrow p$;
- if negative, set $b \leftarrow p$.



> ***Note:***
> *only one evaluation of the function f per iteration.*

3 This reduces the length of the uncertainty interval by a factor of $\frac{1}{2}$ at each iteration. So, setting $x_n = p$, the error after n iterations satisfies

$$|x_n - x^*| \leq \frac{b-a}{2^n}$$

- x^* : the exact root in $[a, b]$
- n : number of bisections (first midpoint is x_1)
- $b - a$: initial interval length

> **Warning: Different Indexing**
>
> Some texts index the first midpoint as x_0 (not x_1). Then the error bound becomes
>
> $$|x_n - x^*| \leq \frac{b-a}{2^{n+1}}$$
>
> Relation between the two notations: x_n (here) $= x_{n+1}$ (in that convention).

4 **Stopping criterion:** given (absolute) tolerance $\texttt{atol}$, require

$$\frac{b-a}{2^n} \leq \texttt{atol}$$

5 This allows ***a priori*** determination of the number of iterations n.

To achieve accuracy $\texttt{atol}$, it suffices to take

$$n \geq \left\lceil \log_2 \left(\frac{b-a}{\texttt{atol}} \right) \right\rceil$$

> **Example**
>
> If $[a, b] = [1, 2]$, then $b - a = 1$ and
>
> $$|x_n - x^*| \leq \frac{1}{2^n} = 2^{-n}.$$

Key takeaway: Each bisection *halves* the interval length, and the error bound decreases as $\frac{1}{2^n}$.

Fixed point iteration

This is an intuitively appealing approach which often leads to simple algorithms for complicated problems.

1 **Write given problem**

$$f(x) = 0$$

as

$$g(x) = x,$$

so that $f(x^*) = 0$ iff $g(x^*) = x^*$.

2 **Iterate:**

$$x_{k+1} = g(x_k), \quad k = 0, 1, \ldots,$$

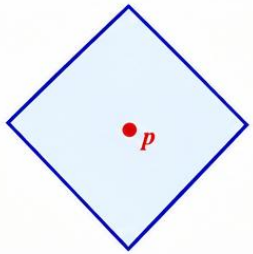
starting with guess x_0.

It's all in the choice of the function g.

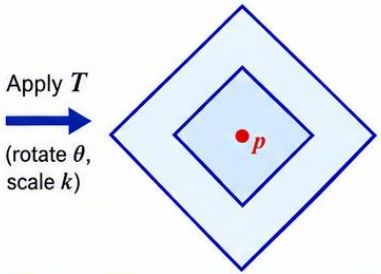
Fixed Point Iteration: Repeated Mapping of a Region

- A mapping T is applied repeatedly. Each time the region is rotated by the same angle θ and scaled by the same factor $0 < k < 1$.
- All images share the same point p (the fixed point), which remains unchanged: $T(p) = p$.

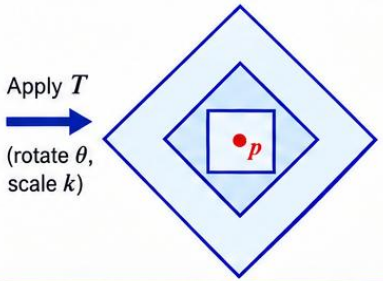
0) Original region D_0 (scale 1, angle 0°)



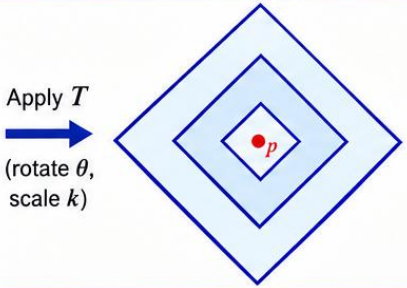
1) First mapping: $D_1 = T(D_0)$ (scale k, angle 45°)



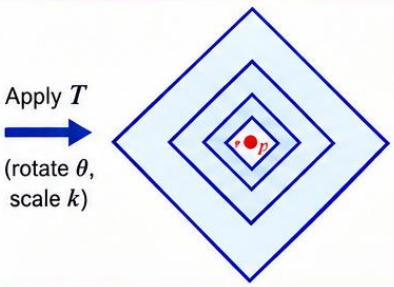
2) Second mapping: $D_2 = T(D_1)$ (scale k^2, angle 90°)



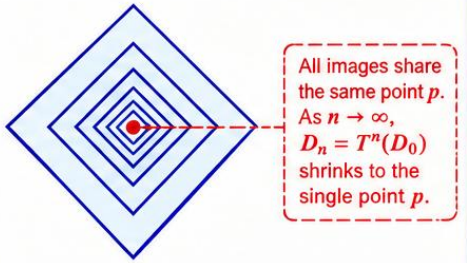
3) Third mapping: $D_3 = T(D_2)$ (scale k^3, angle 135°)



4) Fourth mapping: $D_4 = T(D_3)$ (scale k^4, angle 180°)



5) After many mappings …



If T is a contraction $(0 < k < 1)$, then $T^n(D)$ shrinks to the fixed point p.
For any starting point x_0, the iteration $x_{n+1} = T(x_n)$ converges to p.

Point iteration view



$x_0 \to x_1 = T(x_0) \to x_2 = T(x_1) \to x_3 = T(x_2) \to \cdots \to p$

Choosing the function g

Note: There are many possible choices g for the given f: this is a family of methods.

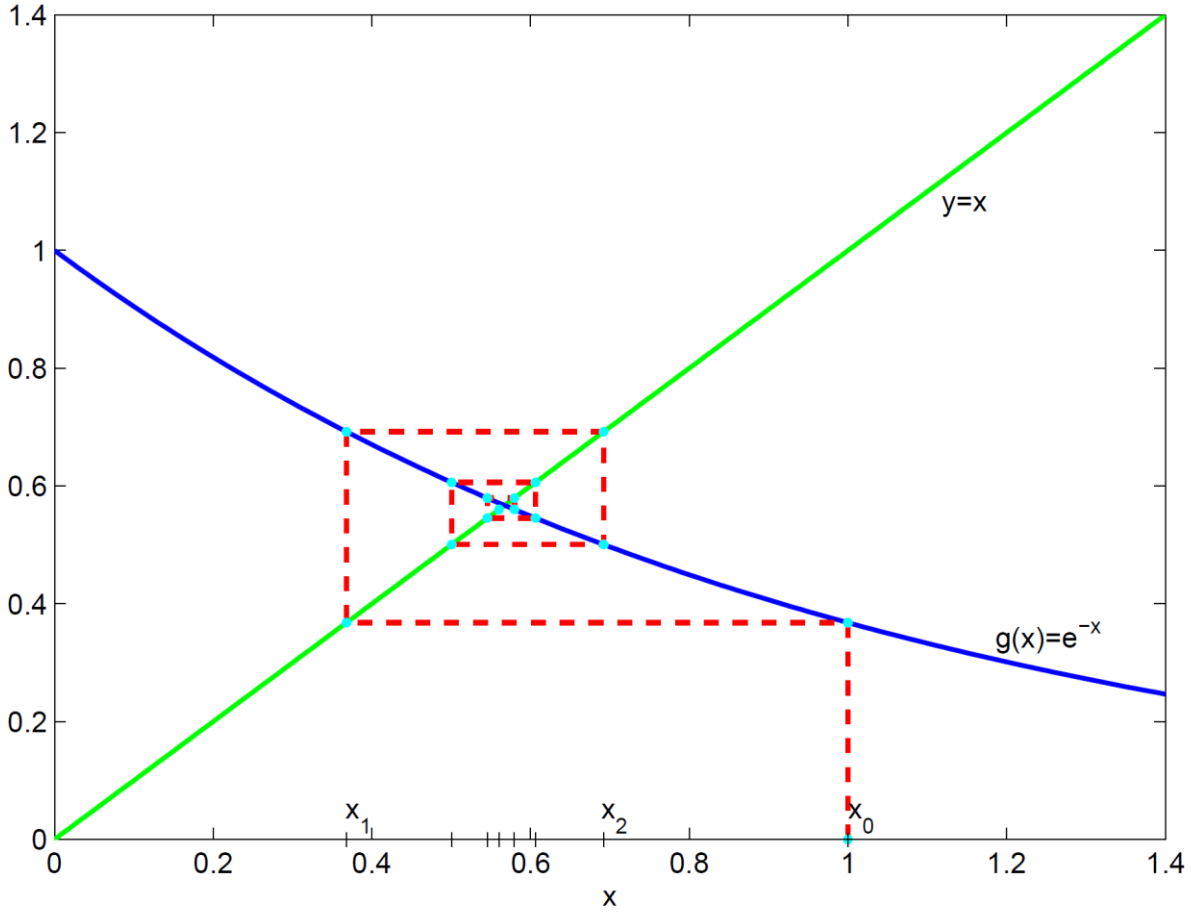
Examples:

	Choice for $g(x)$	Iteration $x_{k+1} = g(x_k)$	Comments
1	$g(x) = x - f(x)$	$x_{k+1} = x_k - f(x_k)$	Simple to implement.
2	$g(x) = x + 2f(x)$	$x_{k+1} = x_k + 2f(x_k)$	Also simple.
3	$g(x) = x - \dfrac{f(x)}{f'(x)}$	$x_{k+1} = x_k - \dfrac{f(x_k)}{f'(x_k)}$	Has potential for fast convergence (we'll see later).

Our goals for the resulting method: Be simple | Converge | Do it rapidly

The performance of fixed point iteration depends critically on the choice of g.

Graphical illustration, $x = e^{-x}$, starting from $x_0 = 1$



Fixed Point Theorem

Informal idea: If the function g maps the interval into itself and is not "too steep" (its slope is less than 1 in magnitude), then it has exactly one fixed point in that interval.

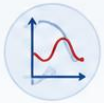
1. If $g \in C[a, b]$, $g(a) \geq a$ and $g(b) \leq b$, then there exists a fixed point x^* in the interval $[a, b]$.

2. If, in addition, the derivative g' exists and there is a constant $\rho < 1$ such that

$$|g'(x)| \leq \rho \qquad \forall\, x \in (a, b),$$

3. then the fixed point x^* is unique in this interval.

Key Conditions



Mapping into the interval: $g(a) \geq a,\ g(b) \leq b$

g'

Bounded slope: $|g'(x)| \leq \rho < 1$



Result: Exactly one fixed point x^*

Why it matters: This theorem guarantees that the iteration $x_{k+1} = g(x_k)$ converges to that unique fixed point for any starting value in $[a, b]$.

Convergence of fixed point iteration

- Assuming $\rho < 1$ as in the fixed point theorem, obtain

$$|x_{k+1} - x^*| = |g(x_k) - g(x^*)| \leq \rho |x_k - x^*|.$$

- This is a **contraction** by factor ρ.
- So

$$|x_{k+1} - x^*| \leq \rho |x_k - x^*| \leq \rho^2 |x_{k-1} - x^*| \leq \cdots \leq \rho^{k+1} |x_0 - x^*| \to 0.$$



The smaller ρ, the faster the convergence.

Fixed-point iterations for $x^3 + 4x^2 - 10 = 0$

We rewrite the equation $x^3 + 4x^2 - 10 = 0$ in the form $x = g(x)$ to obtain fixed-point iterations.

a — Isolate x

$$x = g_1(x) = x - x^3 - 4x^2 + 10$$

From: Move all terms except x to the right.

b — Divide by x

$$x = g_2(x) = \left(\frac{10}{x} - 4x\right)^{1/2}$$

From: $x^3 + 4x^2 = 10 \Rightarrow x^2(x+4) = 10 \Rightarrow x = \sqrt{\frac{10}{x+4}}$.

c — Isolate x^3

$$x = g_3(x) = \frac{1}{2}(10 - x^3)^{1/2}$$

From: $x^3 = 10 - 4x^2$.

d — Factor x^2

$$x = g_4(x) = \left(\frac{10}{4+x}\right)^{1/2}$$

From: $x^2(x+4) = 10 \Rightarrow x^2 = \frac{10}{x+4} \Rightarrow x = \sqrt{\frac{10}{4+x}}$.

e — Rational form

$$x = g_5(x) = x - \frac{x^3 + 4x^2 - 10}{3x^2 + 8x}$$

From: Divide both sides by $3x^2 + 8x$ and solve for x.

Fixed-Point Iteration: Comparison of Five Choices of g.

With $p_0 = 1.5$, the table below shows the iterates produced by five different fixed-point maps g for the equation $x^3 + 4x^2 - 10 = 0$.

n	(a)	(b)	(c)	(d)	(e)
0	1.5	1.5	1.5	1.5	1.5
1	−0.875	0.8165	1.286953768	1.348399725	1.373333333
2	6.732	2.9969	1.402540804	1.367376372	1.365262015
3	−469.7	$(-8.65)^{1/2}$	1.345458374	1.364957015	1.365230014
4	1.03×10^8	—	1.375170253	1.365264748	1.365230013
5	—	—	1.360094193	1.365225594	—
6	—	—	1.367846968	1.365230576	—
7	—	—	1.363887004	1.365229942	—
8	—	—	1.365916734	1.365230022	—
9	—	—	1.364878217	1.365230012	—
10	—	—	1.365410062	1.365230014	—
15	—	—	1.365223680	1.365230013	—
20	—	—	1.365230236	—	—
25	—	—	1.365230006	—	—
30	—	—	1.365230013	—	—

What do we learn?

- (a) diverges rapidly
- (b) becomes invalid
- (c) converges, but slowly
- (d) converges quickly
- (e) converges very quickly

Rate of Convergence

1. Suppose we want $|x_k - x^*| \approx 0.1|x_0 - x^*|$.

2. Since $|x_k - x^*| \leq \rho^k |x_0 - x^*|$, want
$$\rho^k \approx 0.1.$$

3. i.e., $k \log_{10} \rho \approx -1.$

4. Define the **rate of convergence** as
$$rate = -\log_{10} \rho.$$

The rate is the number of decimal digits gained in the error approximately every iteration.

5. Then it takes about $k = [1/rate]$ iterations to reduce the error by more than an order of magnitude (a factor of 10).

Summary

- If an iterative method is linearly convergent with constant ρ $(0 < \rho < 1)$:
$$|x_k - x^*| \leq \rho^k |x_0 - x^*|$$
- Then the error behaves like ρ^k.
- The rate
$$rate = -\log_{10} \rho$$
measures how fast the error decreases.
- Smaller $\rho \rightarrow$ larger rate $\rightarrow$ faster convergence.

Examples

Linear convergence
$\rho = 0.1$ $\quad rate = 1$
(~1 digit / iteration)

Linear convergence
$\rho = 0.01$ $\quad rate = 2$
(~2 digits / iteration)

Quadratic convergence
$\rho^k \sim C2^{-2^k} \Rightarrow$ rate grows very rapidly

Key takeaway: The rate of convergence quantifies how quickly accuracy improves.
A rate of r means we gain about r **correct decimal digits** of accuracy per iteration.

24

Return to cosh Example

1 **Bisection method**

$$rate = -\log_{10} 0.5 \approx 0.3010 \approx 0.3 \quad \Rightarrow \quad k = 4.$$

About **4 iterations** are needed to reduce the error by one order of magnitude.

2 **Fixed point method (root x_1^*)**

For the root x_1^*, $\rho \approx 0.31$, so

$$rate = -\log_{10} 0.31 \approx 0.5086 \approx 0.5 \quad \Rightarrow \quad k = 2.$$

About **2 iterations** are needed to reduce the error by one order of magnitude.

3 **Fixed point method (root x_2^*)**

For the root x_2^*, $\rho > 1$, so

$$rate = -\log_{10}(\rho) < 0 \quad \Rightarrow \quad no \text{ convergence.}$$

Since $\rho > 1$, the error grows and the method **does not converge**.

Key takeaway: The rate of convergence $rate = -\log_{10} \rho$ tells us how fast the error decreases.
Larger rate $\Rightarrow$ fewer iterations to gain one order of magnitude in accuracy.

Newton's method

This fundamentally important method is everything that bisection is not, and vice versa:

- Not so simple
- Not very safe or robust
- Requires more than continuity on f
- Fast
- Automatically generalizes to systems

Derivation

1 By Taylor series



Expanding $f(x)$ about x_k,

$$f(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{f''(\xi(x))}{2}(x - x_k)^2.$$

2 At the root x^*



Setting $x = x^*$ (so that $f(x^*) = 0$), we get

$$0 = f(x_k) + f'(x_k)(x^* - x_k) + \mathcal{O}\left((x^* - x_k)^2\right).$$

3 Neglect the nonlinear term



Dropping the higher-order term and solving for x^* gives

$$0 = f(x_k) + f'(x_k)(x^* - x_k).$$

Replacing x^* by the next iterate x_{k+1} yields the iteration step

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \qquad k = 0, 1, 2, \ldots$$

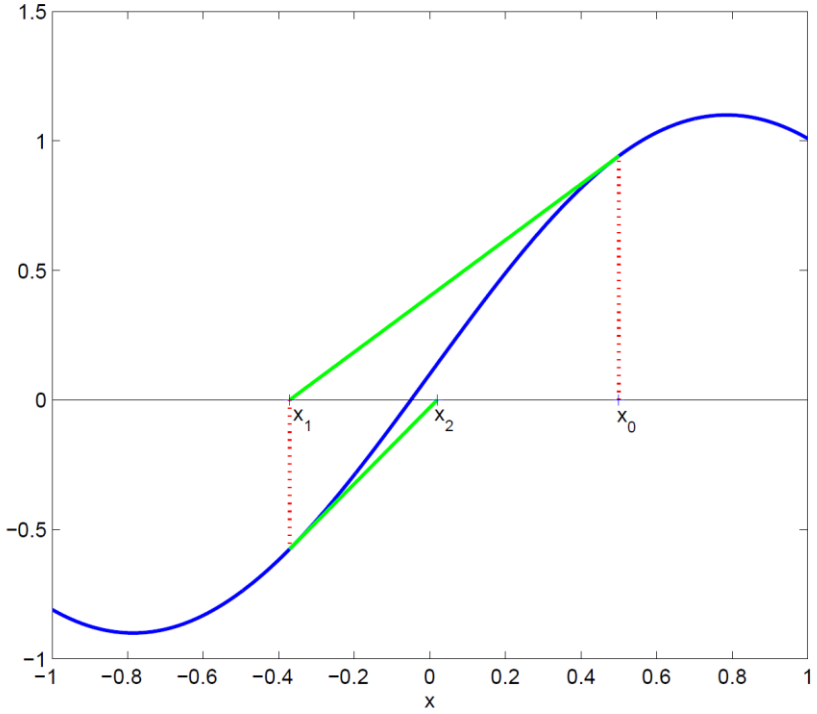
i **This is Newton's method.** It uses the tangent line at x_k to approximate the root.

Algorithm 2.3: NEWTON'S METHOD

To find a solution to $f(x) = 0$ given an initial approximation p_0:

INPUT initial approximation p_0; tolerance *TOL*; maximum number of iterations N_0.
OUTPUT approximate solution p or message of failure.
Step 1 Set $i = 1$.
Step 2 While $i \leq N_0$ do Steps 3–6.
    Step 3 Set $p = p_0 - f(p_0)/f'(p_0)$. (*Compute* p_i.)
    Step 4 If $|p - p_0| < TOL$ then
        OUTPUT (p); (*The procedure was successful.*)
        STOP.
    Step 5 Set $i = i + 1$.
    Step 6 Set $p_0 = p$. (*Update* p_0.)
Step 7 OUTPUT ('The method failed after N_0 iterations, $N_0 =$', N_0);
    (*The procedure was unsuccessful.*)
    STOP.

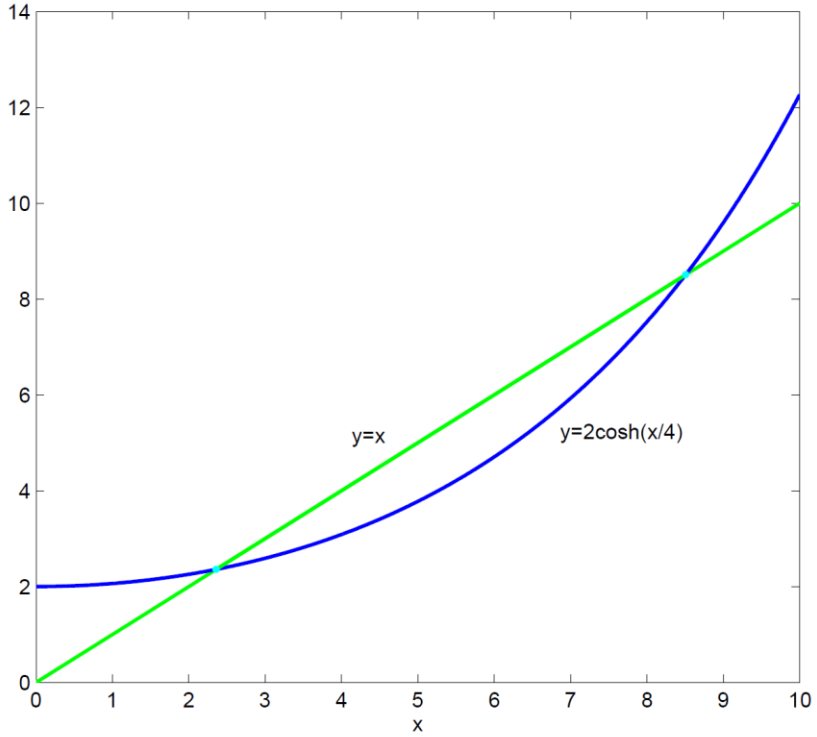
A geometric interpretation



Next iterate is x-intercept of the tangent line to f at current iterate.

Example: cosh with two roots

$$f(x) = g(x) - x, \quad g(x) = 2\cosh(x/4)$$



Example: cosh with two roots

- The function

$$f(x) = 2\cosh(x/4) - x$$

has two solutions in the interval $[2, 10]$.

- Newton's iteration is

$$x_{k+1} = x_k - \frac{2\cosh(x_k/4) - x_k}{0.5\sinh(x_k/4) - 1}.$$

- For absolute tolerance 1.e-8:
  - Starting from $x_0 = 2$ requires 4 iterations to reach x_1^*.
  - Starting from $x_0 = 4$ requires 5 iterations to reach x_1^*.
  - Starting from $x_0 = 8$ requires 5 iterations to reach x_2^*.
  - Starting from $x_0 = 10$ requires 6 iterations to reach x_2^*.
- Tracing the iteration's progress:

k	0	1	2	3	4	5
$f(x_k)$	-4.76e-1	8.43e-2	1.56e-3	5.65e-7	7.28e-14	1.78e-15

- Note that the number of significant digits essentially doubles at each iteration (until the 5th, when roundoff error takes over).

Speed of Convergence

Let $\{x_k\}$ be the sequence of approximations produced by an iterative method that converges to the solution x^*.

A given method is said to be:

1 Linearly convergent

if there is a constant ρ with $0 \leq \rho < 1$ such that

$$|x_{k+1} - x^*| \leq \rho |x_k - x^*|,$$

for all k sufficiently large.

Intuition

The error is reduced by at least a constant factor ρ at each iteration.



Steady (linear) decay

2 Quadratically convergent

if there is a constant $M > 0$ such that

$$|x_{k+1} - x^*| \leq M |x_k - x^*|^2,$$

for all k sufficiently large.

Intuition

The number of correct digits roughly doubles at each iteration.



Very fast (quadratic) decay

3 Superlinearly convergent

if there is a sequence $\{\rho_k\}$ with $\rho_k \to 0$ such that

$$|x_{k+1} - x^*| \leq \rho_k |x_k - x^*|,$$

for all k sufficiently large.

Intuition

The reduction factor improves to zero; faster than linear, but not necessarily quadratic.



Faster than linear (not necessarily quadratic)

Key takeaway: Quadratic convergence (as in Newton's method near a simple root) is much faster than linear convergence and is the reason for its efficiency in practice.

Convergence Theorem for Newton's Method

Theorem (Local quadratic convergence)

If $f \in C^2[a,b]$ and there exists a root x^* in $[a,b]$ such that

$$f(x^*) = 0, \quad f'(x^*) \neq 0,$$

then there is a number $\delta > 0$ such that, starting with x_0 from anywhere in the neighborhood $[x^* - \delta, x^* + \delta]$, Newton's method converges **quadratically** to x^*.

Idea of proof

1. Expand $f(x^*)$ in terms of a Taylor series about x_k:

$$f(x^*) = f(x_k) + f'(x_k)(x^* - x_k) + \frac{1}{2}f''(\xi_k)(x^* - x_k)^2, \quad \xi_k \text{ between } x_k \text{ and } x^*.$$

2. Divide by $f'(x_k)$, rearrange, and replace $x_k - \frac{f(x_k)}{f'(x_k)}$ by x_{k+1}:

$$x^* - x_{k+1} = -\frac{f''(\xi_k)}{2f'(x_k)}(x^* - x_k)^2.$$

3. Find the relation between the errors $e_{k+1} = x_{k+1} - x^*$ and $e_k = x_k - x^*$:

$$e_{k+1} = -\frac{f''(\xi_k)}{2f'(x_k)}e_k^2 \quad \Rightarrow \quad \text{quadratic convergence.}$$

Key takeaway: Near a simple root ($f(x^*) = 0,\ f'(x^*) \neq 0$), Newton's method converges **quadratically**, meaning the number of correct digits roughly doubles at each iteration.

Fixed-Point Iteration vs Newton's Method

Problem: For $f(x) = \cos x - x = 0$, approximate the root using (a) fixed-point iteration and (b) Newton's method.

Initial guess: $p_0 = \frac{\pi}{4} \approx 0.7853981635$

True root: $x^* \approx 0.7390851332$

A Fixed-point iteration

$p_{n+1} = \cos(p_n)$ Simple, but only linearly convergent here.

n	p_n
0	0.7853981635
1	0.7071067810
2	0.7602445972
3	0.7246674808
4	0.7487198858
5	0.7325608446
6	0.7434642113
7	0.7361282565

After 7 iterations, the sequence is still oscillating around the root.

B Newton's method

$p_{n+1} = p_n + \frac{\cos p_n - p_n}{1 + \sin p_n}$ Quadratic convergence near the root.

n	p_n
0	0.7853981635
1	0.7395361337
2	0.7390851781
3	**0.7390851332**
4	**0.7390851332**

By iteration 3: $p_3 = 0.7390851332$

Takeaway: Both methods approach the same root, but Newton's method reaches high accuracy in far fewer iterations.

Secant Method

Why Secant? Newton's method requires evaluating the derivative. The Secant method avoids an explicit derivative evaluation.

1 Approximate the derivative near the root

Use the slope of the secant line through the two most recent iterates:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$$

2 Substitute this approximation into Newton's update

This gives the Secant iteration:

$$x_{k+1} = x_k - \frac{f(x_k)\,(x_k - x_{k-1})}{f(x_k) - f(x_{k-1})}, \qquad k = 1, 2, \ldots$$

✓ **Important:** the method needs two initial values, x_0 and x_1.

Unlike Newton's method, no explicit derivative is required.

→ **a two-step iterative method**

Secant Illustration

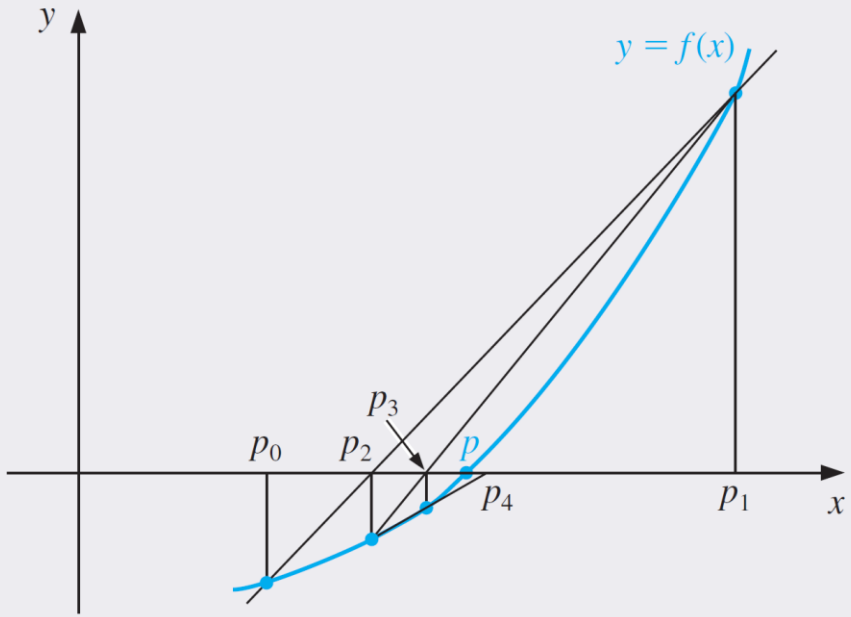


Figure: Figure 2.9

Secant Method vs Newton's Method

Use the Secant method to find a solution to $x = \cos x$, and compare the approximations with those given in Example 1 which applied Newton's method.

Table 2.5

Newton	
n	p_n
0	0.7853981635
1	0.7395361337
2	0.7390851781
3	0.7390851332
4	0.7390851332

Secant	
n	p_n
0	0.5
1	0.7853981635
2	0.7363841388
3	0.7390581392
4	0.7390851493
5	0.7390851332

Takeaway: Both methods converge to approximately 0.7390851332. Newton converges in fewer iterations, while the secant method does not require explicit derivative evaluation.

Newton's Method as a Fixed Point Iteration

1. If $g'(x^*) \neq 0$ then fixed point iteration converges **linearly**, as discussed before, with convergence rate $\rho > 0$.

2. Newton's method can be written as a fixed point iteration with

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

$\Rightarrow$ From this we get $g'(x^*) = 0$,

3. In such a situation the fixed point iteration may converge **faster** than linearly; indeed, Newton's method converges **quadratically** under appropriate conditions.

Key Takeaway: Newton's method is a special fixed point iteration with $g'(x^*) = 0$, which explains its **quadratic convergence**.

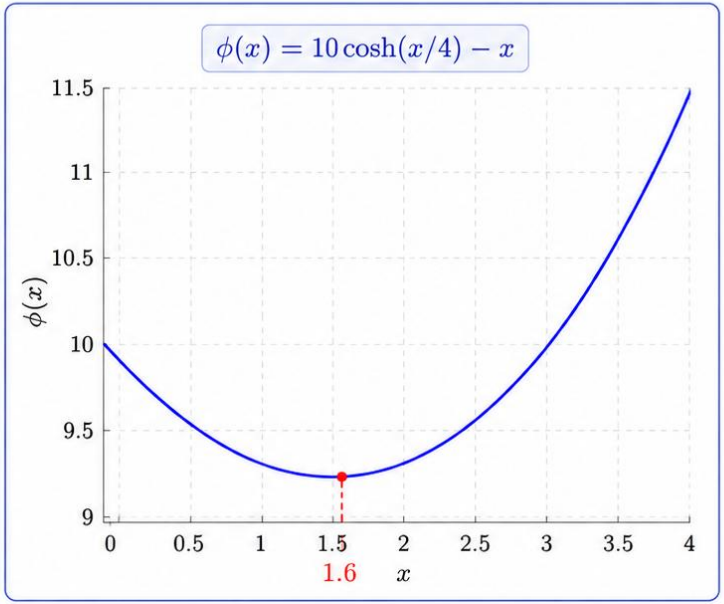
Minimizing a Function in One Variable

Here, we consider the simplest situation of minimizing a smooth function in one variable.

- **Example:** find $x = x^*$ that minimizes

$$\phi(x) = 10\cosh(x/4) - x.$$

- From the figure (right), this function **has no zeros** but does appear to have one minimizer around $x = 1.6$.



Key takeaway: Minimization does not require finding zeros.
We look for the point where the function reaches its **lowest value**.

39

Conditions for Optimum and Algorithm

1. Necessary condition for an optimum

Suppose $\phi \in C^2$ and denote $f(x) = \phi'(x)$. Then a zero of f is a critical point of ϕ, i.e., where

$$\phi'(x^*) = 0.$$

- To be a minimizer or a maximizer, it is necessary for x^* to be a critical point.

2. Sufficient condition for an optimum

A critical point x^* is a **minimizer** if also $\phi''(x^*) > 0$.

3. Algorithm for finding a minimizer

1. Find the roots of $\phi'(x)$ (critical points).
2. For each root x^*, compute $\phi''(x^*)$.
3. If $\phi''(x^*) > 0$, then x^* is a **minimizer**.

Note Rather than finding all roots of ϕ' first and checking the minimum condition later, one can proceed more carefully and wisely, e.g., by sticking to steps that **decrease** $\phi(x)$.

False Position

To find a solution to $f(x) = 0$ for a continuous function f on the interval $[p_0, p_1]$, where $f(p_0)$ and $f(p_1)$ have opposite signs:

INPUT: initial approximations p_0, p_1; tolerance TOL; maximum number of iterations N_0.
OUTPUT: approximate solution p or message of failure.

Step 1 Set $i = 2$;
$q_0 = f(p_0)$;
$q_1 = f(p_1)$.

Step 2 While $i \leq N_0$ do Steps 3–7.

Step 3 Set $p = p_1 - q_1 (p_1 - p_0)/(q_1 - q_0)$. *(Compute p_i.)*

Step 4 If $|p - p_1| < TOL$ then
OUTPUT (p); *(The procedure was successful.)*
STOP.

Step 5 Set $i = i + 1$;
$q = f(p)$.

Step 6 If $q \cdot q_1 < 0$ then set $p_0 = p_1$;
$q_0 = q_1$.

Step 7 Set $p_1 = p$;
$q_1 = q$.

Step 8 OUTPUT ('Method failed after N_0 iterations, N_0 =', N_0);
(The procedure unsuccessful.)
STOP.

Secant - Method of False Position Illustration

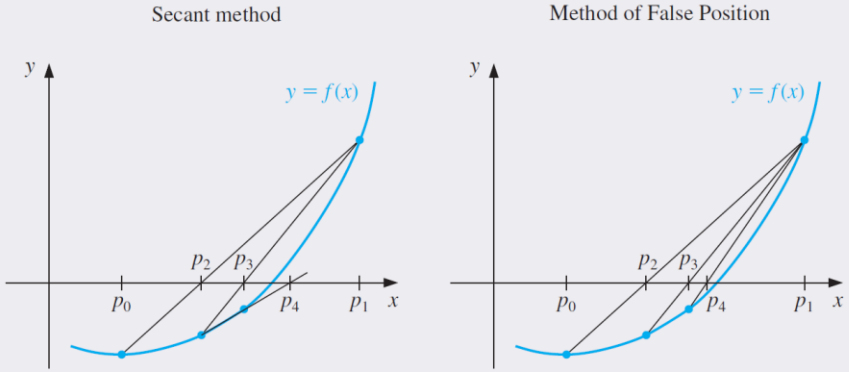


Figure: Figure 2.10

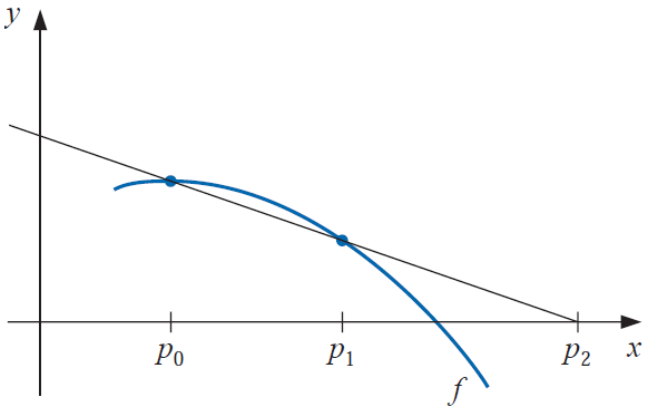
False Position Method: Comparison Example

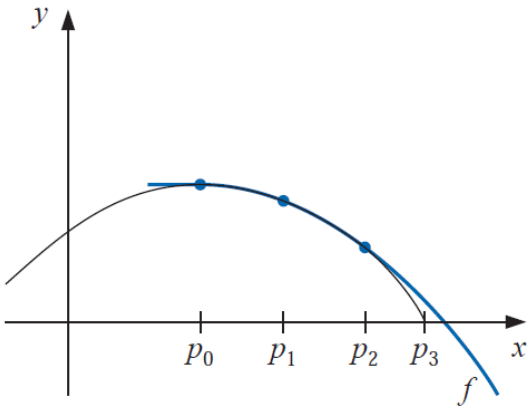
Use the method of False Position to find a solution to $x = \cos x$, and compare the approximations with those given in Example 1 which applied fixed-point iteration and Newton's method, and to those found in Example 2 which applied the Secant method.

n	False Position p_n	Secant p_n	Newton p_n
0	0.5	0.5	0.7853981635
1	0.7853981635	0.7853981635	0.7395361337
2	0.7363841388	0.7363841388	0.7390851781
3	0.7390581392	0.7390581392	0.7390851332
4	0.7390848638	0.7390851493	0.7390851332
5	0.7390851305	0.7390851332	
6	0.7390851332		

All three methods converge to approximately 0.7390851332. | **Fastest:** Newton › **Next:** Secant › **Slowest:** False Position

Müller's method is similar to the Secant method. But whereas the Secant method uses a line through two points on the curve to approximate the root, Müller's method uses a parabola through three points on the curve for the approximation.





Algorithm 2.8: MÜLLER'S METHOD

To find a solution to $f(x) = 0$ given three approximations, p_0, p_1, and p_2:

INPUT p_0, p_1, p_2; tolerance TOL; maximum number of iterations N_0.
OUTPUT approximate solution p or message of failure.

Step 1 Set $h_1 = p_1 - p_0$; $h_2 = p_2 - p_1$; $\delta_1 = (f(p_1) - f(p_0))/h_1$;
$\delta_2 = (f(p_2) - f(p_1))/h_2$; $d = (\delta_2 - \delta_1)/(h_2 + h_1)$; $i = 3$.

Step 2 While $i \leq N_0$ do Steps 3–7.

Step 3 $b = \delta_2 + h_2 d$; $D = (b^2 - 4f(p_2)d)^{1/2}$. (*May require complex arithmetic.*)

Step 4 If $|b - D| < |b + D|$ then set $E = b + D$
else set $E = b - D$.

Step 5 Set $h = -2f(p_2)/E$; $p = p_2 + h$.

Step 6 If $|h| < TOL$ then
OUTPUT (p); (*The procedure was successful.*)
STOP.

Step 7 Set $p_0 = p_1$; (*Prepare for next iteration.*)
$p_1 = p_2$; $p_2 = p$; $h_1 = p_1 - p_0$; $h_2 = p_2 - p_1$; $\delta_1 = (f(p_1) - f(p_0))/h_1$;
$\delta_2 = (f(p_2) - f(p_1))/h_2$; $d = (\delta_2 - \delta_1)/(h_2 + h_1)$; $i = i + 1$.

Step 8 OUTPUT ('Method failed after N_0 iterations, $N_0 =$', N_0);
(*The procedure was unsuccessful.*)
STOP.

Error Analysis for Iterative Methods

Linear Convergence

The error decreases proportionally:

$$|e_{n+1}| \approx C|e_n|$$

Model sequence: $\{p_n\}_{n=0}^{\infty} = (0.5)^n$

Quadratic Convergence

The error decreases quadratically:

$$|e_{n+1}| \approx C|e_n|^2$$

Model sequence: $\{\tilde{p}_n\}_{n=0}^{\infty} = (0.5)^{2^{n-1}}$

n	Linear Convergence Sequence $\{p_n\}_{n=0}^{\infty} = (0.5)^n$	Quadratic Convergence Sequence $\{\tilde{p}_n\}_{n=0}^{\infty} = (0.5)^{2^{n-1}}$
1	5.0000×10^{-1}	5.0000×10^{-1}
2	2.5000×10^{-1}	1.2500×10^{-1}
3	1.2500×10^{-1}	7.8125×10^{-3}
4	6.2500×10^{-2}	3.0518×10^{-5}
5	3.1250×10^{-2}	4.6566×10^{-10}
6	1.5625×10^{-2}	1.0842×10^{-19}
7	7.8125×10^{-3}	5.8775×10^{-39}

After 7 iterations: Linear error $\approx 7.8125 \times 10^{-3}$ | Quadratic error $\approx 5.8775 \times 10^{-39}$ (many orders of magnitude smaller)

Takeaway: Quadratic convergence reduces the error **exponentially** fast—far more rapidly than linear convergence.

Theorem (2.8): Convergence of Fixed-Point Iteration (Linear Case)

Theorem (2.8)

Let $g \in C[a,b]$ be such that $g(x) \in [a,b]$ for all $x \in [a,b]$.

Suppose, in addition, that g' is continuous on (a,b) and there exists a constant k with $0 \leq k < 1$ such that

$$|g'(x)| \leq k, \qquad \text{for all } x \in (a,b).$$

If $g'(p) \neq 0$, then for any number $p_0 \neq p$ in $[a,b]$, the sequence

$$p_n = g(p_{n-1}), \qquad \text{for } n \geq 1,$$

converges only **linearly** to the unique fixed point p in $[a,b]$.

Key takeaway: If $|g'(x)|$ is uniformly bounded by a constant $k < 1$, then the fixed-point iteration converges to the unique fixed point, but the convergence is **linear (slow)**.

Theorem (2.9): Quadratic Convergence of Fixed-Point Iteration

Theorem (2.9)

Let p be a **solution** of the equation $x = g(x)$.
Suppose that $g'(p) = 0$ and g'' is continuous with $|g''(x)| \leq M$
on an open interval I containing p.

Then there exists a $\delta > 0$ such that, for any $p_0 \in [p - \delta, p + \delta]$,
the sequence defined by $p_n = g(p_{n-1})$, when $n \geq 1$,
converges at least **quadratically** to p.
Moreover, for sufficiently large values of n,

$$|p_{n+1} - p| \leq \frac{M}{2} |p_n - p|^2.$$

Key takeaway: If $g'(p) = 0$ and g'' is bounded near p, then fixed-point iteration converges **quadratically** to p from any starting point **sufficiently close** to p.

Definition (2.10: Multiplicity)

A solution p of $f(x) = 0$ is a **zero of multiplicity** m of f if for $x \neq p$, we can write $f(x) = (x - p)^m q(x)$, where $\lim_{x \to p} q(x) \neq 0$.

Theorem (2.11)

The function $f \in C^1[a, b]$ has a **simple zero** at p in (a, b) if and only if $f(p) = 0$, but $f'(p) \neq 0$.

Theorem (2.12)

The function $f \in C^m[a, b]$ has a zero of multiplicity m at p in (a, b) if and only if

$$0 = f(p) = f'(p) = f''(p) = \cdots = f^{(m-1)}(p), \quad \text{but} \quad f^{(m)}(p) \neq 0.$$

Zeros and Multiplicity

Definition (2.10): Multiplicity

A number p is a **zero of multiplicity m** of f if

$$f(x) = (x-p)^m q(x) \qquad \text{with} \qquad \lim_{x \to p} q(x) \neq 0.$$

In words: f has a factor $(x-p)^m$, and the remaining factor $q(x)$ does not vanish at p.

Theorem (2.11): Simple zero

The function $f \in C^1[a,b]$ has a **simple zero** at $p \in (a,b)$ if and only if

$$f(p) = 0, \qquad f'(p) \neq 0.$$

Theorem (2.12): Zero of multiplicity m

The function $f \in C^m[a,b]$ has a **zero of multiplicity m** at $p \in (a,b)$ if and only if

$$f(p) = f'(p) = f''(p) = \cdots = f^{(m-1)}(p) = 0, \qquad f^{(m)}(p) \neq 0.$$

Equivalently, the first $m-1$ derivatives vanish at p, and the m-th derivative does not.

Takeaway

- **Simple zero**: the graph crosses the $x-$axis at p (slope nonzero).
- **Multiplicity** $m > 1$: the graph touches the $x-$axis and is flatter near p.

Example: Comparing Newton's Methods

In Section 2.2 we found that a zero of $f(x) = x^3 + 4x^2 - 10 = 0$ is $p = 1.36523001$.
Here we will compare convergence for a simple zero using both Newton's method and the modified Newton's method listed in Eq. (2.13). Let p_k denote the k-th iterate.

(i) Newton's method

$$p_k = p_{k-1} - \frac{p_{k-1}^3 + 4p_{k-1}^2 - 10}{3p_{k-1}^2 + 8p_{k-1}}.$$

(ii) Modified Newton's method (Eq. 2.13)

$$p_k = p_{k-1} - \frac{(p_{k-1}^3 + 4p_{k-1}^2 - 10)(3p_{k-1}^2 + 8p_{k-1})}{(3p_{k-1}^2 + 8p_{k-1})^2 - (p_{k-1}^3 + 4p_{k-1}^2 - 10)(6p_{k-1} + 8)}.$$

Initial guess: $p_0 = 1.5$

Newton's method			Modified Newton's method			Actual zero
$p_1 = 1.37333333,$	$p_2 = 1.36526201,$	$p_3 = 1.36523001.$	$p_1 = 1.35689898,$	$p_2 = 1.36519585,$	$p_3 = 1.36523001.$	$p = 1.36523001$

Conclusion: Both methods are **rapidly convergent** to the actual zero, which is given (to 8 decimal places) as p_3 by both methods.
Note, however, that in the case of a simple zero the original Newton's method requires **substantially less computation** per iteration.

NAZARBAYEV
UNIVERSITY

